Relatório do Projeto

Representação de formas geométricas através de listas ligadas em Java

Unidade curricular: Estruturas de Dados
Tecnologia: Java 17+
Sistema de build: Maven
Arquitetura principal: `Node`, `Connection` e `Shape`
Data: outubro de 2026
Autor: *[preencher nome do aluno]*

Resumo

Este projeto implementa um sistema de representação de três formas geométricas: triângulos, retângulos e círculos. O sistema foi desenvolvido em Java, utilizando Maven, e tem como objetivo demonstrar a construção manual de listas ligadas e a navegação entre nós.

A arquitetura do domínio é baseada em três objetos principais: `Node`, que representa um ponto ou vértice; `Connection`, que representa uma ligação entre dois nós; e `Shape`, que representa a forma geométrica. As restantes classes são auxiliares e servem para implementar as listas ligadas, aplicar o Strategy Pattern, criar as formas, percorrer a estrutura e apresentar uma representação visual no terminal.

O sistema impede a criação de tipos de formas diferentes dos três permitidos. Para os polígonos, as ligações formam um ciclo fechado. Como um círculo não possui vértices matemáticos nem arestas retas, é representado por um nó de referência no centro e por um raio guardado na forma.

---

1. Introdução

As estruturas ligadas são estruturas de dados fundamentais porque permitem relacionar objetos através de referências. Ao contrário de uma estrutura baseada exclusivamente em posições consecutivas, uma lista ligada guarda em cada elemento uma referência para o elemento seguinte.

Neste projeto, essa ideia é aplicada à representação de formas geométricas. Os pontos da forma são objetos `Node`, enquanto as ligações entre esses pontos são objetos `Connection`. Uma `Shape` agrupa os nós, as ligações e o tipo da forma.

O projeto foi desenvolvido com uma preocupação especial de manter visível o funcionamento interno da estrutura. Por esse motivo, não são utilizadas classes como `ArrayList` ou `LinkedList` para guardar os elementos principais. As listas de nós e de ligações foram implementadas manualmente.

---

2. Objetivos

Os principais objetivos do projeto são:

- representar exclusivamente triângulos, retângulos e círculos;
- utilizar listas ligadas implementadas manualmente;
- representar os pontos através de objetos `Node`;
- representar as ligações através de objetos `Connection`;
- agrupar a estrutura numa classe `Shape`;
- permitir identificar `current`, `previous` e `next` durante um percurso;
- tratar corretamente referências `null`;
- aplicar o Strategy Pattern na construção das formas;
- utilizar Maven para compilar e executar o projeto;
- apresentar as formas visualmente no terminal;
- rejeitar formas que não estejam entre os três tipos permitidos.

---

3. Requisitos funcionais

O sistema deve cumprir as seguintes regras:

3.1 Tipos permitidos

Só são permitidos os seguintes valores:

```
public enum ShapeType {
    TRIANGLE,
    RECTANGLE,
    CIRCLE
}
```

Qualquer valor diferente, como `PENTAGON`, `SQUARE` ou `HEXAGON`, deve provocar uma exceção.

3.2 Triângulo

Um triângulo deve possuir exatamente:

- 3 objetos `Node`;
- 3 objetos `Connection`.

As ligações fecham o ciclo da forma:

```
T1 → T2 → T3 → T1
```

Além disso, os três pontos não podem ser colineares.

3.3 Retângulo

Um retângulo deve possuir exatamente:

- 4 objetos `Node`;
- 4 objetos `Connection`.

As ligações são:

```
R1 → R2 → R3 → R4 → R1
```

A largura e a altura devem ser valores positivos.

3.4 Círculo

Um círculo não possui vértices matemáticos nem lados retos. Para manter uma representação coerente com a estrutura do projeto, utiliza-se:

- um `Node` de referência identificado por `C`;
- as coordenadas do centro guardadas nesse nó;
- o raio guardado no objeto `Shape`;

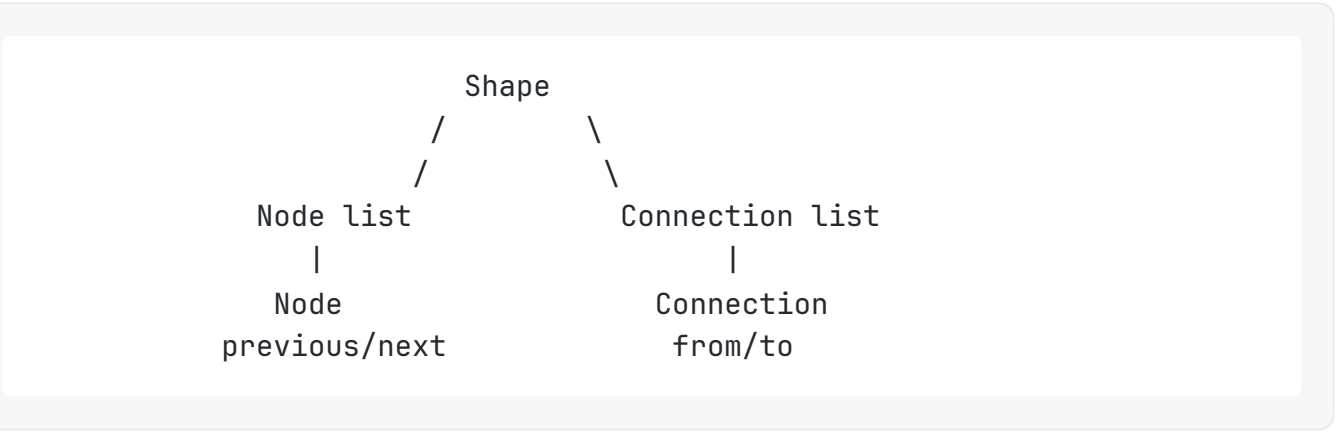
- zero objetos `Connection`;
- `C.previous == null`;
- `C.next == null`.

O nó central é uma âncora de representação, não um vértice da circunferência.

---

4. Arquitetura do sistema

A arquitetura principal é composta pelos seguintes objetos:



Os três objetos principais do domínio são:

1. `Node`;
2. `Connection`;
3. `Shape`.

As listas ligadas são estruturas auxiliares que guardam instâncias desses objetos.

4.1 Classe `Node`

A classe `Node` representa um ponto ou vértice. Os seus principais atributos são:

```
private final String id;
private final double x;
private final double y;
private Node previous;
private Node next;
```

O identificador e as coordenadas são definidos na criação. As referências `previous` e `next` começam com o valor `null`:

```
this.previous = null;
this.next = null;
```

Os métodos `setPrevious` e `setNext` têm acesso de pacote. Desta forma, apenas as classes da estrutura podem alterar as ligações dos nós.

4.2 Classe `Connection`

A classe `Connection` representa uma ligação dirigida entre dois `Node`:

```
private final Node from;
private final Node to;
```

Uma ligação entre `N1` e `N2` significa:

```
Connection(N1, N2)
N1.next = N2
N2.previous = N1
```

A classe rejeita ligações que contenham nós `null`, porque uma ligação sem origem ou destino não é válida.

4.3 Classe `Shape`

A classe `Shape` é o terceiro objeto principal. Não existem subclasses de domínio chamadas `Triangle`, `Rectangle` ou `Circle`. O tipo é guardado no atributo `ShapeType`.

A classe mantém:

```
private final ShapeType type;
private final LinkedNodeList nodes;
private final LinkedConnectionList connections;
private double radius;
```

O método `connect` é responsável por atualizar os nós e inserir uma nova `Connection` na lista ligada:

```
from.setNext(to);
to.setPrevious(from);
connections.add(new Connection(from, to));
```

A própria classe `Shape` valida o número de nós e ligações de acordo com o seu tipo.

---

5. Implementação das listas ligadas

5.1 `LinkedNodeList`

A classe `LinkedNodeList` guarda os objetos `Node`. Para isso, possui uma classe interna de entrada:

```
private static final class NodeEntry {
    private final Node value;
    private NodeEntry nextEntry;
}
```

Cada `NodeEntry` guarda um nó e uma referência para a entrada seguinte. A inserção no final da lista utiliza a referência `last`, evitando percorrer novamente todos os elementos.

A operação de acesso por índice começa no primeiro elemento e percorre as entradas uma a uma até encontrar a posição pedida. Assim, o funcionamento da lista ligada fica explícito.

5.2 `LinkedConnectionList`

A classe `LinkedConnectionList` aplica a mesma lógica aos objetos `Connection`. A sua entrada interna é `ConnectionEntry`, que possui:

```
private final Connection value;
private ConnectionEntry nextEntry;
```

Não são utilizadas coleções prontas da biblioteca Java. As listas são, portanto, adequadas ao objetivo académico de demonstrar a estrutura de dados manualmente.

5.3 Diferença entre listas e ligações geométricas

É importante distinguir duas utilizações da palavra “ligação”:

- `nextEntry` liga entradas internas da lista ligada;
- `Node.next` e `Node.previous` ligam nós da estrutura geométrica.

Por exemplo, a lista pode guardar `T1`, `T2` e `T3` através de `NodeEntry`. Ao mesmo tempo, a geometria usa `T1.next = T2`, `T2.next = T3` e `T3.next = T1`.

---

6. Navegação com `current`, `previous` e `next`

Durante o percurso, a variável `current` representa o nó que está a ser visitado.

Para obter os nós vizinhos:

```
Node previous = current.getPrevious();
Node next = current.getNext();
```

O percurso é efetuado seguindo `next`:

```
Node current = shape.getStartNode();

for (int step = 0;
     step < shape.getNodeCount() && current != null;
     step++) {

    Node previous = current.getPrevious();
    Node next = current.getNext();

    // mostrar os três valores
    current = next;
}
```

6.1 Triângulo

O percurso do triângulo é:

```
current = T1, previous = T3, next = T2
current = T2, previous = T1, next = T3
current = T3, previous = T2, next = T1
```

Depois de três passos, o percurso regressaria a `T1`. Por isso, o método limita o número de passos ao número de nós.

6.2 Retângulo

O percurso do retângulo é:

```
current = R1, previous = R4, next = R2
current = R2, previous = R1, next = R3
current = R3, previous = R2, next = R4
current = R4, previous = R3, next = R1
```

Também é um ciclo fechado.

6.3 Círculo

No círculo, existe apenas o nó de referência `C`:

```
current = C
previous = null
next = null
```

Como não existe `next`, o percurso termina sem gerar um erro.

---

7. Strategy Pattern

O Strategy Pattern foi utilizado para separar a construção de cada tipo de forma.

7.1 Interface da estratégia

A interface `ShapeCreationStrategy` define o comportamento comum:

```
public interface ShapeCreationStrategy {
    ShapeType supportedType();
    int expectedParameterCount();
    Shape create(double... values);
}
```

7.2 Strategies concretas

Existem três estratégias:

Estratégia	Tipo criado	Parâmetros
`TriangleCreationStrategy`	`TRIANGLE`	6
`RectangleCreationStrategy`	`RECTANGLE`	4
`CircleCreationStrategy`	`CIRCLE`	3

As estratégias não criam subclasses. Todas criam o mesmo tipo de objeto principal, `Shape`, e preenchem as listas com `Node` e `Connection`.

7.3 `ShapeFactory` como Contexto

A `ShapeFactory` funciona como o Contexto do padrão. O seu fluxo é:

```
ShapeFactory.create("triangle", valores)
        |
        v
TriangleCreationStrategy
        |
        v
Shape(TRIANGLE)
        |
        +-- Node
        +-- Node
        +-- Node
        +-- Connection
        +-- Connection
        +-- Connection
```

Primeiro, o texto recebido é convertido para `ShapeType`. Depois é escolhida a Strategy correspondente. Por fim, a quantidade de parâmetros é validada antes de a Strategy construir a forma.

Esta separação torna o código mais organizado: a fábrica decide qual algoritmo utilizar, enquanto cada Strategy conhece os detalhes da construção da sua forma.

---

8. Validação e tratamento de erros

O projeto possui várias validações.

8.1 Tipo desconhecido

Se for pedido um tipo desconhecido:

```
ShapeFactory.create("pentagono", ...);
```

é lançada uma `IllegalArgumentException` com uma mensagem que informa que só são aceites `TRIANGLE`, `RECTANGLE` e `CIRCLE`.

8.2 Tipo `null`

Um tipo `null` também é rejeitado:

```
ShapeFactory.create(null, ...);
```

8.3 Quantidade de parâmetros

Cada Strategy indica o número de parâmetros esperado. A fábrica compara esse valor com a quantidade recebida antes de chamar `create`.

8.4 Valores geométricos

Também são rejeitados:

- identificadores de nós nulos ou vazios;
- ligações com nós nulos;
- largura ou altura não positivas no retângulo;
- raio não positivo no círculo;
- estruturas com uma quantidade incorreta de nós ou ligações.

---

9. Maven e organização do projeto

O projeto segue a estrutura Maven convencional:

```
geometry-linked-list-java/
├── pom.xml
├── README.md
└── src/main/java/
    ├── model/
    │   ├── Node.java
    │   ├── Connection.java
    │   ├── Shape.java
    │   └── ShapeType.java
    ├── repository/
    │   ├── LinkedNodeList.java
    │   └── LinkedConnectionList.java
    ├── strategy/
    │   ├── ShapeCreationStrategy.java
    │   ├── TriangleCreationStrategy.java
    │   ├── RectangleCreationStrategy.java
    │   └── CircleCreationStrategy.java
    ├── control/
    │   ├── ShapeFactory.java
    │   └── Main.java
    └── view/
        ├── ShapeNavigator.java
        └── TerminalShapeRenderer.java
```

A organização por packages separa o projeto por responsabilidades: `model` contém os três objetos principais; `repository` contém as listas ligadas; `strategy` contém as estratégias de criação; `control` contém a fábrica e o `Main`; e `view` contém a navegação e a apresentação no terminal.

O ficheiro `pom.xml` define:

- `groupId`: `pt.academico`;
- `artifactId`: `geometry-linked-list-java`;
- versão `1.0-SNAPSHOT`;
- Java `17` como versão de compilação;
- `maven-compiler-plugin`;
- `exec-maven-plugin` para executar `Main`.

Os comandos de execução são:

```
mvn clean compile
mvn exec:java
```

---

10. Visualização no terminal

A classe `TerminalShapeRenderer` fornece uma representação ASCII das formas. Esta funcionalidade é apenas visual e não substitui a estrutura de dados.

Triângulo

```
      /\
     /  \
    /    \
   /      \
  /________\
```

Retângulo

```
+------------+
|            |
|            |
|            |
+------------+
```

Círculo

```
      *****
   **       **
  **           **
 *               *
 *               *
  **           **
    **       **
      *****
```

A renderização é feita pelo `ShapeNavigator` antes da impressão das ligações e dos percursos.

---

11. Testes e resultados

A execução foi validada com Maven através dos comandos:

```
mvn clean compile
mvn exec:java
```

11.1 Resultado do triângulo

```
Nós: 3 | Ligações: 3
T1 → T2
T2 → T3
T3 → T1
```

O percurso apresentado foi:

```
T3 | T1 | T2
T1 | T2 | T3
T2 | T3 | T1
```

Cada linha apresenta:

```
previous | current | next
```

11.2 Resultado do retângulo

```
Nós: 4 | Ligações: 4
R1 → R2
R2 → R3
R3 → R4
R4 → R1
```

A estrutura contém quatro nós e quatro ligações, formando um ciclo fechado.

11.3 Resultado do círculo

```
Nós: 1 | Ligações: 0
null | C | null
```

Este resultado confirma que o círculo utiliza um nó de referência e não possui ligações retas.

11.4 Rejeição de forma inválida

O programa também testa a criação de um pentágono:

```
Tentativa rejeitada: Forma não permitida: pentagono.
Apenas TRIANGLE, RECTANGLE e CIRCLE são aceites.
```

A tentativa é rejeitada pela `ShapeFactory` antes de ser criada qualquer forma inválida.

---

12. Análise da solução

12.1 Vantagens

A solução apresenta as seguintes vantagens:

- separa claramente os três objetos principais;
- mostra manualmente o funcionamento das listas ligadas;
- permite navegar nos dois sentidos através de `previous` e `next`;
- impede tipos de formas desconhecidos através de `ShapeType`;
- separa a criação das formas através do Strategy Pattern;
- valida quantidades de nós, ligações e parâmetros;
- trata corretamente o caso especial do círculo;
- pode ser compilada e executada com comandos Maven simples;
- possui uma representação visual no terminal que facilita a demonstração.

12.2 Complexidade das operações

Considerando `n` elementos numa lista ligada:

Operação	Complexidade
Inserção no fim, mantendo `last`	`O(1)`
Acesso por índice	`O(n)`
Procura de um nó na lista	`O(n)`
Percurso completo dos nós	`O(n)`

O acesso por índice é linear porque, numa lista ligada, é necessário começar no primeiro elemento e seguir os ponteiros até à posição pretendida.

12.3 Limitações

A representação ASCII não pretende ser uma representação geométrica à escala. Serve apenas para mostrar visualmente o tipo de forma no terminal.

Além disso, o círculo é representado por um nó de referência central, porque um círculo matemático não possui um conjunto finito de vértices. Esta é uma decisão de modelação adequada ao requisito de usar nós, mas o nó `C` não deve ser interpretado como um vértice da circunferência.

---

13. Conclusão

O projeto cumpre o objetivo de representar três formas geométricas através de uma estrutura baseada em listas ligadas. A arquitetura principal utiliza os três objetos pedidos: `Node`, `Connection` e `Shape`.

Os nós possuem referências `previous` e `next`, permitindo percorrer as estruturas e identificar o nó anterior, atual e seguinte. As ligações são representadas por objetos `Connection`, que mantêm a relação entre os nós e atualizam as referências de navegação.

A utilização de uma única classe `Shape`, combinada com `ShapeType`, evita a criação de classes de domínio desnecessárias para cada forma. O Strategy Pattern organiza a criação dos diferentes tipos sem alterar o modelo principal.

O projeto também cumpre os requisitos de Maven, de implementação manual das listas ligadas, de validação dos tipos permitidos e de apresentação das formas no terminal. Os testes de execução confirmam as quantidades de nós e ligações, o tratamento de `null` no círculo, o percurso dos polígonos e a rejeição de tipos inválidos.

Anexo A — Comandos de execução

Na pasta raiz do projeto:

```
mvn clean compile
mvn exec:java
```

Para consultar o código-fonte:

```
src/main/java/
```

Para consultar a saída de demonstração:

```
demo-output.txt
```